

|                                            |                                  |
|--------------------------------------------|----------------------------------|
| Universidad Galileo                        | Guatemala 31 de agosto 2026      |
| Facultad: FISICC                           | Alumno: Luis Cordova             |
| Curso: Microprocesadores                   | Carnet: 23012122                 |
| Sección: A                                 | Hora de Laboratorio: 1:00 a 1:50 |
| Auxiliar: Brandon Eduardo Mendoza Espinoza | Día de Laboratorio: Lunes        |

## Laboratorio #3 Programación en C

### Resumen:

En esta práctica se implementaron dos funciones en C para el microcontrolador ARM Cortex-M4, usando Visual Studio Code con el Keil Studio Pack y el compilador Arm Compiler 6.

La primera, `myGCD`, procesa una cadena de texto con números separados por comas. Recorre la cadena carácter por carácter, extrae cada valor y calcula el máximo común divisor aplicando el algoritmo de Euclides.

La segunda, `myCombinations`, recibe un conjunto definido por una cadena y genera todas las combinaciones de  $n$  elementos. El resultado lo escribe en un arreglo que la función llamante le pasa por referencia. Para ello se usó un arreglo de punteros que apuntan a los elementos del conjunto, y un puntero doble que va moviendo el cursor de escritura entre llamadas.

La verificación se hizo con el simulador Arm Fixed Virtual Platform, mirando las variables en la ventana Watch y la salida de `printf` en la consola de depuración.

### Objetivos:

- **myGCD:** extraer los números de una cadena recorriéndola con punteros y calcular el MCD con el algoritmo de Euclides.
- **myCombinations:** generar combinaciones usando paso por referencia, un arreglo de punteros para los elementos y un puntero doble para ir escribiendo el resultado.
- **Verificación:** probar el programa en el simulador FVP, inspeccionando variables y punteros con las herramientas de depuración.

## Teoría:

El lenguaje C proporciona un control fino sobre la memoria, algo fundamental en sistemas embebidos como el ARM Cortex-M4, donde los recursos son escasos y el programador necesita saber exactamente dónde se almacena cada dato.

Un puntero es una variable que guarda la dirección de memoria de otra variable, en lugar de almacenar un valor directamente. El operador & devuelve la dirección de una variable y el operador \* accede al contenido de esa dirección. Cuando se usa paso por referencia, lo que se envía a la función es la dirección de la variable, no una copia de su valor; así, cualquier modificación dentro de la función afecta directamente a la variable original [2].

Si una función necesita modificar el puntero que recibe, y no solo el dato al que apunta, se utiliza un puntero doble. El operador & eleva el tipo de dato en un nivel, por lo que la dirección de una variable de tipo char \* corresponde a char \*\*. Por otro lado, al incrementar un puntero, este avanza el tamaño del tipo al que apunta [3], y un arreglo de punteros permite acceder a varias cadenas dentro de un mismo bloque de memoria sin necesidad de duplicarlas [6].

En C, una cadena de caracteres es un arreglo de tipo char que termina con el carácter nulo '\0'. Este carácter no se imprime y sirve como indicador de final, ya que las funciones de la biblioteca estándar van leyendo la memoria hasta encontrarlo. Esto permite almacenar varias cadenas independientes dentro de un mismo arreglo, separándolas con caracteres nulos [4].

El algoritmo de Euclides calcula el máximo común divisor de dos números enteros basándose en que este es igual al MCD del menor de ellos y el resto de su división. El proceso se repite reemplazando los operandos hasta que el residuo es cero; el valor que queda es el MCD buscado [1].

Por último, una combinación es una selección en la que no necesariamente participan todos los elementos del conjunto, no se considera el orden y no hay repetición de elementos. Su número se obtiene con la fórmula  $C(k,n) = k! / (n! (k - n)!)$ ; para un conjunto de tres elementos tomados de dos en dos, el resultado son tres combinaciones [5].

## Resultados prácticos:

A continuación, se presentan las capturas obtenidas durante el desarrollo y la ejecución de la práctica.

```
Lab3_LuisCordova_23012122 > C main.c
1  #include "RTE_Components.h"
2  #include CMSIS_device_header
3
4  #include <stdio.h>
5
6  // Operaciones
7  int myGCD(const char *set);
8  void myCombinations(int n, const char *set, char *comb);
9
10 //Datos
11 static const char numbers[] = "10, 60, 48";
12 static const char setA[] = "A = {rojo, verde, azul}";
13
14 // salidas
15 char comb_n1[512]; // n = 1
16 char comb_n2[512]; // n = 2
17 char comb_n3[512]; // n = 3
18
19
```

Ilustración 1- declaración de cabeceras, prototipos de las funciones y variables globales del programa.

```
20 int main(void){
21
22     // MCD
23     int result = myGCD(numbers);
24
25
26     myCombinations(1, setA, comb_n1);
27     myCombinations(2, setA, comb_n2);
28     myCombinations(3, setA, comb_n3);
29
30
31     printf("MCD = %d\n", result);
32     printf("n=1: %s\n", comb_n1);
33     printf("n=2: %s\n", comb_n2);
34     printf("n=3: %s\n", comb_n3);
35
36     while (1);
37
38 }
39
```

Ilustración 2- programa principal con las llamadas a myGCD y a myCombinations para n = 1, 2 y 3.

```

42 // Miramos el residuo hasta que de 0
43 static int EuclidesMCD (int a, int b) {
44     while (b != 0) { // si el residuo es 0 se termino
45         int valor = b; // guardamos valor de b
46         b = a % b; // obtenemos el residuo de a/b
47         a = valor; // nuevo valor para a
48     }
49     return a;
50 }
51
52 int myGCD(const char *set) {
53     const char *p = set; // el puntero que va a recorrer la cadena
54     int gcd = 0; // valor para retorno
55
56     while (*p != '\0') { // leer hasta el final
57
58         while (*p != '\0' && (*p < '0' || *p > '9')) { // ver si es un numero, para el parseo
59             p++;
60         }
61         if (*p == '\0') {
62             break;
63         }
64         // realizar el parseo
65         int num = 0;
66         while (*p >= '0' && *p <= '9') { // mientras sea un numero
67             num = num * 10 + (*p - '0'); // convertir de ascii a int
68             p++;
69         }
70
71         // hacer el MCD y guardarlo en el gcd ( comparamos de 2 en 2 )
72         gcd = EuclidesMCD(gcd, num);
73     }
74
75     return gcd; // resultado del MCD de los numeros
76 }

```

Ilustración 3- implementación de la función auxiliar EuclidesMCD y de la función myGCD en main.c

```

// Parte 2 Cardinalidad de n en k
#define Maxima_Cardinalidad 5 // Definimos la Maxima cardinalidad
#define Maximo_N 3 // Definimos el Maximo valor de n

static void appendStr(char **destino, const char *origen) { // str de origen y donde es el destino de el
    char *d = *destino; // recibe el destino
    while (*origen != '\0') { // copiamos todo el origen
        *d++ = *origen++;
    }
    *destino = d; // actualiamos puntero de destino
}

```

Ilustración 4- definición de las constantes del conjunto y de la función appendStr, que recibe un puntero doble.

```
void myCombinations(int n, const char *set, char *comb) {

    static char buffer[128]; // guardamos los nombres ( azul/rojo/verde/ )
    char *punteros[Maxima_Cardinalidad]; // punteros
    int k = 0; // cardinalidad

    if (comb == 0 || set == 0) {
        return;
    }
    comb[0] = '\0'; // preparar el puntero
    const char *p = set; // empezar donde set empieza
    char *w = buffer; // puntero para escribir los punteros
    while (*p != '\0' && *p != '{') { // buscamos el principio ( validaciones )
        p++;
    } if (*p != '{') { return; }
    p++;
    while (*p != '\0' && *p != '}' && k < Maxima_Cardinalidad) {

        while (*p == ' ') { // saltar espacios
            p++;
        }
        punteros[k] = w; // para ver donde seguir con el ptro puntero o donde empezar
        while (*p != '\0' && *p != ',' && *p != '}') { // copiar elemento
            *w++ = *p++;
        }
        while (w > punteros[k] && *(w - 1) == ' ') { // eliminar los espacios en blanco ( si existen )
            w--;
        }
        *w++ = '\0'; // cerrar la cadena ya que no hay espacios en blanco
        k++;
        if (*p == ',') {
            p++;
        }
    }
}
```

Ilustración 5- análisis de la cadena de entrada dentro de myCombinations y llenado del arreglo de punteros.

```
if (n < 1 || n > Maximo_N || n > k) { // validaciones de valores de n
    return;
}

int indice[Maximo_N]; // guarda espacio para combinaciones
char *salida = comb; // salida
int primera = 1; // para la coma
int i, j; // para los ciclos

for (i = 0; i < n; i++) { // se crea la primera combinacion
    indice[i] = i;
}
```

Ilustración 6- validación de los parámetros recibidos y generación de la primera combinación.

```

92 void mycombinations(int n, const char *set, char *comb) {
136     for (i = 0; i < n; i++) { // se crea la primera combinacion
137         indice[i] = i;
138     }
139
140     for (;;) {
141         if (!primera) {
142             appendStr(&salida, ", "); // separar combinacion
143         }
144         primera = 0;
145
146         appendStr(&salida, "("); // empezar con la lave
147         for (i = 0; i < n; i++) {
148             if (i > 0) {
149                 appendStr(&salida, ", "); // separar los nombres adentro de las combinaciones
150             }
151             appendStr(&salida, punteros[indice[i]]); // copiar la palabra en la combinacion
152         }
153         appendStr(&salida, ")"); // cerrar la combinacion
154
155         i = n - 1;
156         while (i >= 0 && indice[i] == k - n + i) { // miramos el valor maximo de la casilla, si lo alcanzo se cambia el anterior
157             i--;
158         }
159         if (i < 0) { // si se termino salir
160             break;
161         }
162         indice[i]++; // siguiente valor de k [0,2] -> [1,2]
163         for (j = i + 1; j < n; j++) { // reiniciamos las de la derecha [0,2] -> [1,0]
164             indice[j] = indice[j - 1] + 1; // no repetidos, siempre el de la derecha mayor
165         }
166     }
167
168     *salida = '\0';

```

Ilustración 7- bucle de generación de las combinaciones en orden lexicográfico.

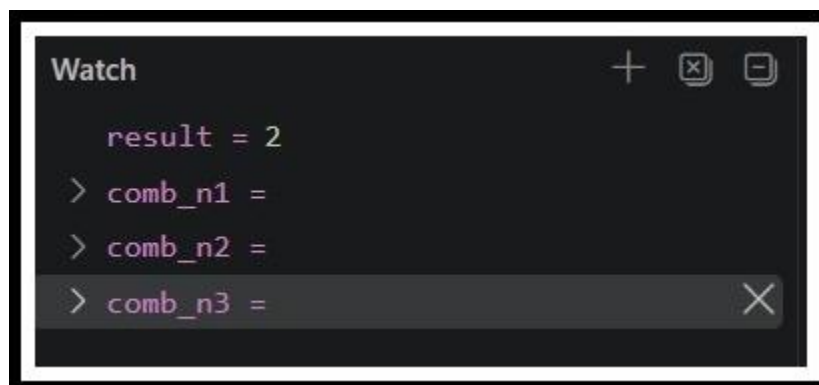


Ilustración 8- ventana Watch con el resultado del máximo común divisor almacenado en la variable result.

```

Watch
result = 2
comb_n1 =
[0] = 123 '{'
[1] = 114 'r'
[2] = 111 'o'
[3] = 106 'j'
[4] = 111 'o'
[5] = 125 '}'
[6] = 44 ','
[7] = 32 ' '
[8] = 123 '{'
[9] = 118 'v'
[10] = 101 'e'
[11] = 114 'r'
[12] = 100 'd'
[13] = 101 'e'
[14] = 125 '}'
[15] = 44 ','
[16] = 32 ' '
[17] = 123 '{'
[18] = 97 'a'
[19] = 122 'z'
[20] = 117 'u'
[21] = 108 'l'
[22] = 125 '}'
[23] = 0 '\0'
[24] = 0 '\0'
[25] = 0 '\0'

```

Ilustración 9- contenido del arreglo comb\_n1, correspondiente a las combinaciones de un elemento.

```

Watch
> comb_n1 =
v comb_n2 =
  [0] = 123 '{'
  [1] = 114 'r'
  [2] = 111 'o'
  [3] = 106 'j'
  [4] = 111 'o'
  [5] = 44 ','
  [6] = 32 ' '
  [7] = 118 'v'
  [8] = 101 'e'
  [9] = 114 'r'
  [10] = 100 'd'
  [11] = 101 'e'
  [12] = 125 '}'
  [13] = 44 ','
  [14] = 32 ' '
  [15] = 123 '{'
  [16] = 114 'r'
  [17] = 111 'o'
  [18] = 106 'j'
  [19] = 111 'o'
  [20] = 44 ','
  [21] = 32 ' '
  [22] = 97 'a'
  [23] = 122 'z'
  [24] = 117 'u'
  [25] = 108 'l'
  [26] = 125 '}'
  [27] = 44 ','
  [28] = 32 ' '

```

Ilustración 10- contenido del arreglo comb\_n2, correspondiente a las combinaciones de dos elementos.



```
result = 2
> comb_n1 =
> comb_n2 =
✓ comb_n3 = [0] = 123 '{'
[1] = 114 'r'
[2] = 111 'o'
[3] = 106 'j'
[4] = 111 'o'
[5] = 44 ','
[6] = 32 ' '
[7] = 118 'v'
[8] = 101 'e'
[9] = 114 'r'
[10] = 100 'd'
[11] = 101 'e'
[12] = 44 ','
[13] = 32 ' '
[14] = 97 'a'
[15] = 122 'z'
[16] = 117 'u'
[17] = 108 'l'
[18] = 125 '}'
[19] = 0 '\0'
[20] = 0 '\0'
[21] = 0 '\0'
[22] = 0 '\0'
[23] = 0 '\0'
[24] = 0 '\0'
[25] = 0 '\0'
```

Ilustración 11- contenido del arreglo comb\_n3, correspondiente a la combinación de tres elementos.

```

[model] MCD = 2
Execution stopped in Privileged Thread mode at 0x0000214C
0x0000214C 32,3 printf("n=1: %s\n", comb_n1);
[model] n=1: {rojo}, {verde}, {azul}
Execution stopped in Privileged Thread mode at 0x0000215A
0x0000215A 33,3 printf("n=2: %s\n", comb_n2);
[model] n=2: {rojo, verde}, {rojo, azul}, {verde, azul}
Execution stopped in Privileged Thread mode at 0x00002168
0x00002168 34,3 printf("n=3: %s\n", comb_n3);
[model] n=3: {rojo, verde, azul}
Execution stopped in Privileged Thread mode at 0x00002174
0x00002174 36,3 while (1);

```

Ilustración 12- salida de la aplicación en el Debug Console mediante la función printf.

## Resultados Teóricos:

### Descripción del código implementado:

El enunciado de la práctica pide implementar una función que calcule el máximo común divisor de un conjunto de enteros recibidos como cadena, y otra que genere todas las combinaciones posibles de n elementos dentro de un conjunto de k elementos, también descrito como texto. Las variables y funciones desarrolladas para cumplir con esto se resumen a continuación.

- **static const char numbers[]:** cadena con los números enteros de prueba separados por comas. Durante la revisión, el auxiliar pidió cambiar el primer valor del conjunto propuesto en el enunciado, quedando la cadena "10, 60, 48", para comprobar que la función opera correctamente con datos distintos a los del ejemplo.
- **static const char setA[]:** cadena que describe el conjunto de elementos con el formato "A = {rojo, verde, azul}" indicado en el enunciado.
- **char comb\_n1[512], comb\_n2[512] y comb\_n3[512]:** arreglos globales que reciben las cadenas de salida generadas para cada valor de n. Se declararon fuera de main porque el espacio de pila reservado por el archivo de dispersión del dispositivo es de 512 bytes, insuficiente para alojar los tres arreglos como variables locales.
- **static int EuclidesMCD(int a, int b):** función auxiliar que calcula el máximo común divisor de dos enteros mediante divisiones sucesivas, reemplazando los operandos hasta que el residuo es cero.
- **int myGCD(const char \*set):** recorre la cadena recibida con un puntero de lectura, descarta los caracteres que no son dígitos, convierte cada secuencia de dígitos a entero y acumula el MCD de los valores obtenidos; retorna el resultado como entero.

- **static void appendStr(char \*\*destino, const char \*origen):** función auxiliar que copia una cadena en la posición actual de escritura y actualiza el puntero de la función llamante. Recibe un puntero doble porque necesita modificar el puntero en sí, no solo el texto al que apunta.
- **void myCombinations(int n, const char \*set, char \*comb):** analiza la cadena que describe el conjunto, valida los parámetros, genera las combinaciones en orden lexicográfico y las escribe en el arreglo apuntado por comb, que se recibe por referencia.

A continuación se describe cada una de las rutinas que componen la aplicación, junto con el análisis de los resultados obtenidos.

### **Declaración de variables globales y prototipos – Ilustración 1**

En la Ilustración 1 se muestran las cabeceras y las declaraciones globales del programa. Se incluye RTE\_Components.h junto con la macro CMSIS\_device\_header, que la herramienta genera automáticamente y que resuelve al archivo de cabecera del dispositivo seleccionado. También se declaran los prototipos de myGCD y myCombinations antes de main, se definen las cadenas constantes numbers y setA, y se reservan los tres arreglos de salida comb\_n1, comb\_n2 y comb\_n3.

### **Programa principal – Ilustración 2**

La rutina principal llama primero a myGCD sobre la cadena numbers y guarda el valor devuelto en la variable local result, que luego se añade a la ventana Watch. Después se hacen tres llamadas a myCombinations con los valores de n permitidos por el enunciado, escribiendo cada resultado en su arreglo correspondiente. Los valores obtenidos se muestran con printf y el programa termina en un bucle infinito, ya que en un sistema embebido no hay un sistema operativo al que retornar.

### **Función myGCD y rutina auxiliar EuclidesMCD – Ilustración 3**

myGCD recorre la cadena con el puntero p, que avanza una posición en cada incremento. El primer bucle interno descarta caracteres que no son dígitos, omitiendo así comas y espacios del formato de entrada. El segundo convierte los dígitos consecutivos a un valor entero usando  $\text{num} = \text{num} * 10 + (*p - '0')$ , que aprovecha la codificación ASCII para obtener el valor numérico de cada carácter. Cada valor extraído se combina con el acumulado mediante EuclidesMCD; la variable gcd se inicializa en cero porque el MCD de cero y cualquier número es ese mismo número, evitando tratar el primer valor como un caso especial.

### **Función appendStr y uso del puntero doble – Ilustración 4**

appendStr es el punto donde se aplica el puntero doble. Copia el texto recibido en la posición apuntada por el puntero de escritura y, antes de terminar, ejecuta  $*destino = d$ , que actualiza el puntero de la función llamante. Si el parámetro se hubiera declarado como puntero simple, la función solo recibiría una copia de la dirección y el avance se perdería al retornar; así, todas las

llamadas escribirían sobre la misma posición y la cadena final solo conservaría el último fragmento copiado. En este archivo también se definen las constantes `Maxima_Cardinalidad` y `Maximo_N`, que corresponden a los límites de cinco elementos por conjunto y tres elementos por combinación establecidos en el enunciado.

### **Análisis de la cadena de entrada – Ilustración 5**

El primer bloque de `myCombinations` convierte la cadena de entrada en una estructura utilizable. Como el parámetro `set` es `const` y está en memoria de solo lectura, los nombres no pueden separarse sobre la cadena original, así que se copian al arreglo `buffer` insertando un carácter nulo después de cada uno. De este modo, un único arreglo contiene varias cadenas independientes. Paralelamente, el arreglo `punteros` guarda la dirección de inicio de cada elemento y la variable `k` lleva el conteo de elementos encontrados, que es la cardinalidad real del conjunto. Los dos bucles que eliminan espacios, uno antes de copiar y otro retrocediendo el puntero de escritura, permiten que la función acepte el conjunto escrito con o sin espacios alrededor de las comas.

### **Validación y primera combinación – Ilustración 6**

Antes de generar resultados, se verifica que `n` esté entre uno y tres y que no supere la cardinalidad del conjunto, ya que no se pueden escoger más elementos de los que este contiene. Si la validación falla, la función retorna sin escribir, y como la cadena de salida se vació al inicio con `comb[0] = '\0'`, la función llamante recibe una cadena vacía sin datos residuales de ejecuciones anteriores. Luego, el arreglo `indice` se inicializa con 0, 1, ..., `n-1`, que es la primera combinación en orden lexicográfico y el punto de partida del generador.

### **Generación de las combinaciones – Ilustración 7**

El bucle principal hace dos cosas en cada iteración: escribe la combinación actual y transforma el arreglo `indice` en la siguiente. La escritura se hace con llamadas sucesivas a `appendStr`, y la bandera `primera` junto con la condición `i > 0` evitan comas al inicio de la cadena o dentro de las llaves. Para avanzar a la siguiente combinación, se recorre `indice` de derecha a izquierda buscando la primera posición que aún puede incrementarse; el valor máximo de cada posición se obtiene con `k - n + i`, que es el valor de esa posición en la última combinación posible. Cuando se encuentra una posición disponible, se incrementa y las posiciones a su derecha se reinician al valor inmediato superior con `indice[j] = indice[j-1] + 1`. Esto asegura que los índices se mantengan en orden estrictamente creciente, condición que por sí sola evita tanto la repetición de elementos dentro de una combinación como la aparición del mismo conjunto en distinto orden. Si ninguna posición puede incrementarse, la variable `i` llega a -1 y el bucle termina.

### **Verificación del máximo común divisor – Ilustración 8**

En la Ilustración 8 se ve la variable result con el valor 2 en la ventana Watch. Este resultado corresponde al conjunto "10, 60, 48" que pidió el auxiliar durante la revisión: el MCD de 10 y 60 es 10, y el de 10 y 48 es 2, así que el valor devuelto es correcto. La verificación confirma que la función analiza bien la cadena de entrada y que el acumulado del algoritmo de Euclides se aplica sobre los tres valores.

### **Verificación de las combinaciones – Ilustraciones 9, 10 y 11**

Las Ilustraciones 9, 10 y 11 muestran el contenido de los arreglos de salida expandidos carácter por carácter en la ventana Watch. En la Ilustración 9 se ve la secuencia para  $n = 1$ , con las tres combinaciones de un elemento; en la Ilustración 10, la de  $n = 2$ , con tres combinaciones de dos elementos; y en la Ilustración 11, la única combinación posible para  $n = 3$ . En las tres capturas se observa que los caracteres nulos aparecen solo al final de la cadena y no entre combinaciones, lo que confirma que el resultado se almacena como una sola cadena continua y no como cadenas separadas.

### **Verificación de la salida por printf – Ilustración 12**

La Ilustración 12 presenta la salida de la aplicación en la Debug Console. Los valores mostrados son  $MCD = 2$ , y las cadenas {rojo}, {verde}, {azul} para  $n = 1$ ; {rojo, verde}, {rojo, azul}, {verde, azul} para  $n = 2$ ; y {rojo, verde, azul} para  $n = 3$ . El número de combinaciones obtenido coincide con lo previsto por la teoría combinatoria: para un conjunto de tres elementos se obtienen tres combinaciones de uno, tres de dos y una de tres, y el formato de la cadena es el que pide el enunciado.

### **Errores y advertencias obtenidos durante la compilación**

Al compilar por primera vez el código base del enunciado, el compilador Arm Compiler 6 reportó el siguiente error:

```
error: call to undeclared function 'myGCD'; ISO C99 and later do not support implicit function declarations [-Wimplicit-function-declaration]
```

El error aparece porque en el código del enunciado myGCD se invoca dentro de main antes de que el compilador conozca su existencia. En versiones antiguas de C esto solo generaba una advertencia, ya que el compilador asumía implícitamente que la función retornaba un entero; sin embargo, a partir de C99 las declaraciones implícitas dejaron de estar permitidas y las versiones recientes del compilador las reportan como error. La corrección fue declarar los prototipos de myGCD y myCombinations antes de main, como se ve en la Ilustración 1. Hecho esto, la compilación del proyecto se completó sin errores ni advertencias.

## Conclusiones:

- Comprendí la diferencia práctica entre un puntero simple y un puntero doble, que hasta esta práctica me parecía solo una complicación de sintaxis. Al recibir un puntero simple, la función trabaja sobre una copia de la dirección y cualquier avance se pierde al retornar; al recibir la dirección del puntero, es posible actualizar la variable original de la función llamante. La cantidad de asteriscos no es arbitraria, sino consecuencia de que el operador & eleva el tipo de dato en un nivel.
- Aprendí que el carácter nulo no es solamente una marca de fin de cadena, sino el mecanismo que permite tener varias cadenas independientes dentro de un mismo arreglo. Gracias a esto, el arreglo de punteros no necesita copiar los nombres del conjunto, solo guardar la dirección donde empieza cada uno, lo cual reduce el uso de memoria en un sistema con recursos limitados.
- Comprobé que en un sistema embebido el tamaño de la pila es una restricción real y no un detalle teórico. Los tres arreglos de salida no cabían en los 512 bytes que reserva el archivo de dispersión del dispositivo, así que tuve que declararlos como variables globales para ubicarlos en la RAM general. Declarados como locales habrían provocado un fallo en tiempo de ejecución difícil de rastrear.
- Concluí que es mejor garantizar una condición por construcción que verificarla después. En el generador de combinaciones no hay ninguna instrucción que compruebe si un elemento se repite; la regla de mantener los índices en orden estrictamente creciente elimina por sí sola tanto las repeticiones como los conjuntos duplicados en distinto orden, lo que da un código más corto y sin necesidad de descartar resultados ya generados.

## **Bibliografía:**

[1] Wikipedia, «Algoritmo de Euclides». [En línea]. Disponible en:

[https://es.wikipedia.org/wiki/Algoritmo\\_de\\_Euclides](https://es.wikipedia.org/wiki/Algoritmo_de_Euclides)

[2] M. E. García, «Guía para el manejo de punteros en C», Prof. Matías E. García. [En línea].

Disponible en:

[https://profmatiasgarcia.com.ar/uploads/tutoriales/Guia\\_para\\_manejo\\_de\\_punteros.pdf](https://profmatiasgarcia.com.ar/uploads/tutoriales/Guia_para_manejo_de_punteros.pdf)

[3] Universidad del País Vasco (UPV/EHU), «Los punteros en C», material docente. [En línea].

Disponible en: [https://lsi.vc.ehu.eus/pablogn/docencia/manuales/C/Punteros\\_en\\_C.pdf](https://lsi.vc.ehu.eus/pablogn/docencia/manuales/C/Punteros_en_C.pdf)

[4] Wikipedia, «Cadenas de texto en C». [En línea]. Disponible en:

[https://es.wikipedia.org/wiki/Cadenas\\_de\\_texto\\_en\\_C](https://es.wikipedia.org/wiki/Cadenas_de_texto_en_C)

[5] Superprof, «Fórmulas de combinatoria», Apuntes de Matemáticas. [En línea]. Disponible en:

<https://www.superprof.es/apuntes/escolar/matematicas/probabilidades/combinatoria/formulas-de-combinatoria.html>

[6] J. Delphin, «Punteros y arreglos», Programación II, Universidad de Los Andes, Núcleo

Universitario Rafael Rangel. [En línea]. Disponible en:

[https://webdelprofesor.ula.ve/nucleotrujillo/jalejandro/Prog2/Unidad1\\_1.pdf](https://webdelprofesor.ula.ve/nucleotrujillo/jalejandro/Prog2/Unidad1_1.pdf)